

```

import static org.junit.Assert.assertEquals;

import org.junit.Test;

import components.sequence.Sequence;

/**
 * JUnit test fixture for {@code Sequence<String>}s constructor and kernel
 * methods.
 *
 * @author Zhuangzhuang He
 */
public abstract class SequenceTest {

    /**
     * Invokes the appropriate {@code Sequence} constructor for the
     * implementation under test and returns the result.
     *
     * @return the new sequence
     * @ensures constructorTest = <>
     */
    protected abstract Sequence<String> constructorTest();

    /**
     * Invokes the appropriate {@code Sequence} constructor for the reference
     * implementation and returns the result.
     *
     * @return the new sequence
     * @ensures constructorRef = <>
     */
    protected abstract Sequence<String> constructorRef();

    /**
     *
     * Creates and returns a {@code Sequence<String>} of the implementation
     * under test type with the given entries.
     *
     * @param args
     *         the entries for the sequence
     * @return the constructed sequence
     * @ensures createFromArgsTest = [entries in args]
     */
    private Sequence<String> createFromArgsTest(String... args) {
        Sequence<String> sequence = this.constructorTest();
        for (String s : args) {
            sequence.add(sequence.length(), s);
        }
        return sequence;
    }

    /**
     *
     * Creates and returns a {@code Sequence<String>} of the reference
     * implementation type with the given entries.
     *
     * @param args
     *         the entries for the sequence
     * @return the constructed sequence
     * @ensures createFromArgsRef = [entries in args]
     */
    private Sequence<String> createFromArgsRef(String... args) {
        Sequence<String> sequence = this.constructorRef();
        for (String s : args) {
            sequence.add(sequence.length(), s);
        }
        return sequence;
    }
}

```

```
/*
 * Constructor tests
 */

@Test
public final void testConstructorEmpty() {
    Sequence<String> s = this.constructorTest();
    Sequence<String> sExpected = this.constructorRef();

    assertEquals(sExpected, s);
}

/*
 * Add tests
 */

@Test
public final void testAddToEmptyAtZero() {
    Sequence<String> s = this.createFromArgsTest();
    Sequence<String> sExpected = this.createFromArgsRef("red");

    s.add(0, "red");

    assertEquals(sExpected, s);
}

@Test
public final void testAddAtFront() {
    Sequence<String> s = this.createFromArgsTest("blue", "green");
    Sequence<String> sExpected = this.createFromArgsRef("red", "blue",
        "green");

    s.add(0, "red");

    assertEquals(sExpected, s);
}

@Test
public final void testAddAtMiddle() {
    Sequence<String> s = this.createFromArgsTest("red", "green");
    Sequence<String> sExpected = this.createFromArgsRef("red", "blue",
        "green");

    s.add(1, "blue");

    assertEquals(sExpected, s);
}

@Test
public final void testAddAtEnd() {
    Sequence<String> s = this.createFromArgsTest("red", "blue");
    Sequence<String> sExpected = this.createFromArgsRef("red", "blue",
        "green");

    s.add(s.length(), "green");

    assertEquals(sExpected, s);
}

/*
 * Remove tests
 */

@Test
public final void testRemoveOnlyEntry() {
    Sequence<String> s = this.createFromArgsTest("red");
    Sequence<String> sExpected = this.createFromArgsRef();

    String removed = s.remove(0);
```

```
        assertEquals("red", removed);
        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveFront() {
        Sequence<String> s = this.createFromArgsTest("red", "blue", "green");
        Sequence<String> sExpected = this.createFromArgsRef("blue", "green");

        String removed = s.remove(0);

        assertEquals("red", removed);
        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveMiddle() {
        Sequence<String> s = this.createFromArgsTest("red", "blue", "green");
        Sequence<String> sExpected = this.createFromArgsRef("red", "green");

        String removed = s.remove(1);

        assertEquals("blue", removed);
        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveEnd() {
        Sequence<String> s = this.createFromArgsTest("red", "blue", "green");
        Sequence<String> sExpected = this.createFromArgsRef("red", "blue");

        String removed = s.remove(s.length() - 1);

        assertEquals("green", removed);
        assertEquals(sExpected, s);
    }

    /*
     * Length tests
     */

    @Test
    public final void testLengthEmpty() {
        Sequence<String> s = this.createFromArgsTest();

        assertEquals(0, s.length());
    }

    @Test
    public final void testLengthNonEmpty() {
        Sequence<String> s = this.createFromArgsTest("red", "blue", "green");

        assertEquals(3, s.length());
    }

    @Test
    public final void testLengthAfterAdd() {
        Sequence<String> s = this.createFromArgsTest("red", "green");

        s.add(1, "blue");

        assertEquals(3, s.length());
    }

    @Test
    public final void testLengthAfterRemove() {
        Sequence<String> s = this.createFromArgsTest("red", "blue", "green");
```

```
        s.remove(1);  
        assertEquals(2, s.length());  
    }  
}
```